TECHNISCHE HOCHSCHULE DEGGENDORF

Programming 1

Orga / Recap

- We all have a solid understanding of
  - variables, types and collections?
  - Loops, conditions
  - Functions & modules
  - Object oriented software development?
- Any questions so far?

Calendar

6./7.10.	Intro, Orga, Hello, World!	8./9.12.	lectures
13./14.10.	Variables, basics	15./16.12.	mid-term
20.21./10.	Collections, Lists, ...	22./23.12.	lectures
27./28.10.	Conditions and Loops	12./13.1.	lectures
3./4.11.	Functions and Modules	18./19.1.	recap
10./11.11.	Classes and Methods	from 26.1.	exams...
17./18.11.	UI + TK basics		
24./25.11.	Spaceship		
01./02.12.	Invaders and Collisions		

GUI / Tkinter

User Interface

What we used so far:

- Console: data=input („give me data: “)

Howto

- Make menues?
- Use mouse?
- ...

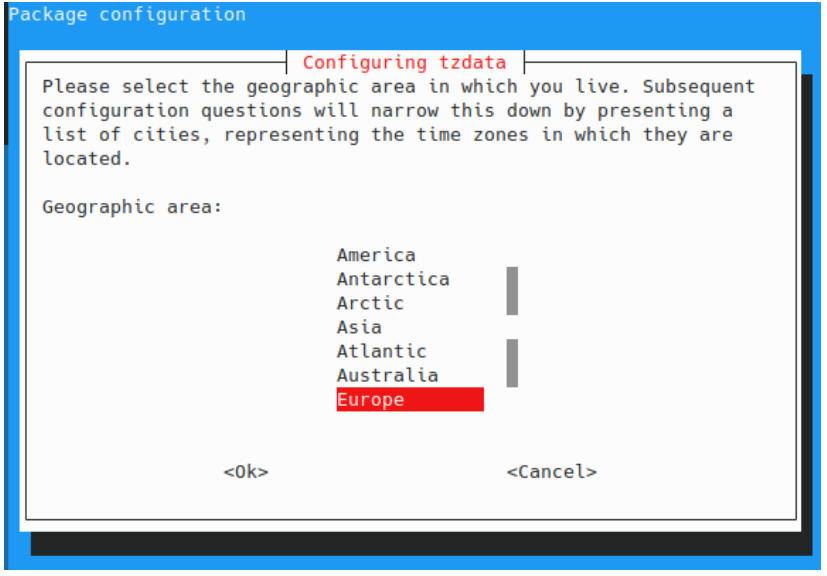
User Interface

What we used so far:

- Console: data=input („give me data: “)

Howto

- Make menues?
- Use mouse?
- …



GUI

- GUI → Graphical User Interface
- In simple words it’s a visual way for users to interact with programs
- It suits windows, buttons, menus and graphics instead of plain text
- Examples : Web browsers, games, mobile apps

Key Components of a GUI

- Windows : Container area that holds other elements
- Widgets : Interactive elements like buttons, text fields, sliders
- Events : User actions like mouse clicks, key presses, mouse movements
- Graphics : Images, shapes, animations that make interfaces engaging

Tkinter

- Tkinter is python’s built in GUI library
- There’s no installation needed since it comes with python
- Perfect for learning GUI programming
- A user can create the following :
  - Windows
  - Canvas (Useful for drawing and games)
  - Buttons

Why Tkinter for games

- Simple to learn
- Good for 2D graphics
- Built–in event handling (keyboard , mouse)
- Components that are used often:
  - Tk() - main window container
  - Canvas - drawing area for graphics and games
  - Event system – Handle keyboard and mouse input
  - Geometry Managers – Control layout of elements

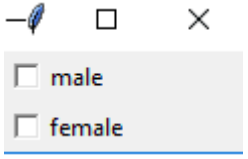
Tkinter Window

```
import tkinter as tk

# Step 1: Create the main application window
root = tk.Tk()

# Step 2: Customize the window
root.title("My First GUI Application")
root.geometry("400x300")  # width x height in pixels
root.resizable(True, True)  # Allow window resizing
root.configure(bg='lightblue')  # Set background color

# Step 3: Start the event loop
root.mainloop()
```

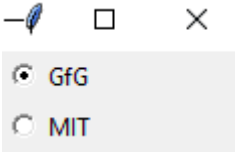
Tk widgets

- Label
- Entry

- CheckButton

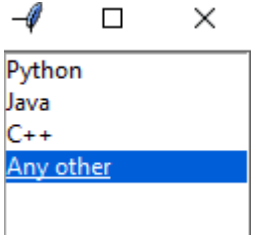


- RadioButton



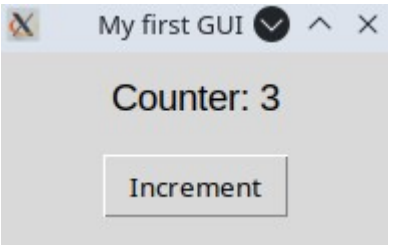
- Listbox

```
from tkinter import *

top = Tk()
Lb = Listbox(top)
Lb.insert(1, 'Python')
Lb.insert(2, 'Java')
Lb.insert(3, 'C++')
Lb.insert(4, 'Any other')
Lb.pack()
top.mainloop()
```



Example

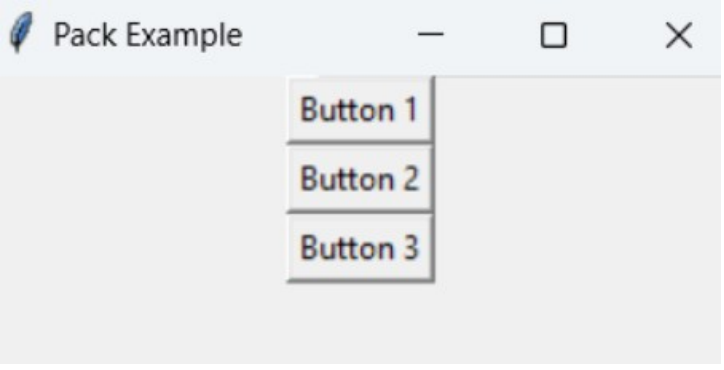
```python
import tkinter as tk

counter = 0
def increment_counter():
    global counter
    counter += 1
    counter_label.config(text=f"Counter: {counter}")

root = tk.Tk()

root.title("My first GUI")
root.geometry("800x600")

counter_label = tk.Label(root, text=f"Counter: {counter}", font=("Arial", 14))
counter_label.pack(pady=10)

increment_button = tk.Button(root, text="Increment", command=increment_counter)
increment_button.pack(pady=5)

root.mainloop()
```



Layouts in tkinter

- Pack() method
  - It organizes the widgets in blocks before placing in the parent widget.
  - Widgets can be packed from the top, bottom, left or right.
  - It can expand widgets to fill the available space or place them in a fixed size.

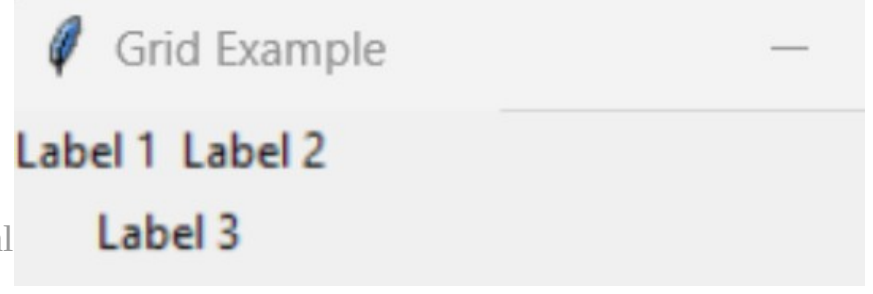
```python
import tkinter as tk

root = tk.Tk()
root.title("Pack Example")

# Create three buttons
button1 = tk.Button(root, text="Button 1")
button2 = tk.Button(root, text="Button 2")
button3 = tk.Button(root, text="Button 3")

# Pack the buttons vertically
button1.pack()
button2.pack()
button3.pack()

root.mainloop()
```



Layouts in tkinter

- grid() method
  - organizes the widgets in grid (table-like structure) before placing in the parent widget.
  - Each widget is assigned a row and column.
  - Widgets can span multiple rows or columns using rowspan and columnspan.

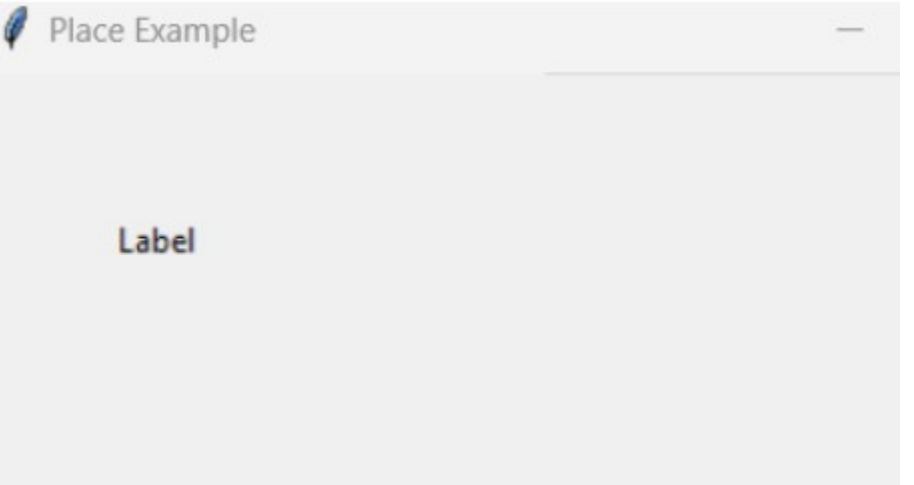
```python
import tkinter as tk

root = tk.Tk()
root.title("Grid Example")

# Create three labels
label1 = tk.Label(root, text="Label 1")
label2 = tk.Label(root, text="Label 2")
label3 = tk.Label(root, text="Label 3")

# Grid the labels in a 2x2 grid
label1.grid(row=0, column=0)
label2.grid(row=0, column=1)
label3.grid(row=1, column=0, columnspan=2)

root.mainloop()
```



Layouts in tkinter

- place() method
  - organizes the widgets by placing them on specific positions directed by the programmer.
  - Widgets are placed at specific x and y coordinates.
  - Sizes and positions can be specified in absolute or relative terms.

```
import tkinter as tk

root = tk.Tk()
root.title("Place Example")

# Create a label
label = tk.Label(root, text="Label")

# Place the label at specific coordinates
label.place(x=50, y=50)

root.mainloop()
```



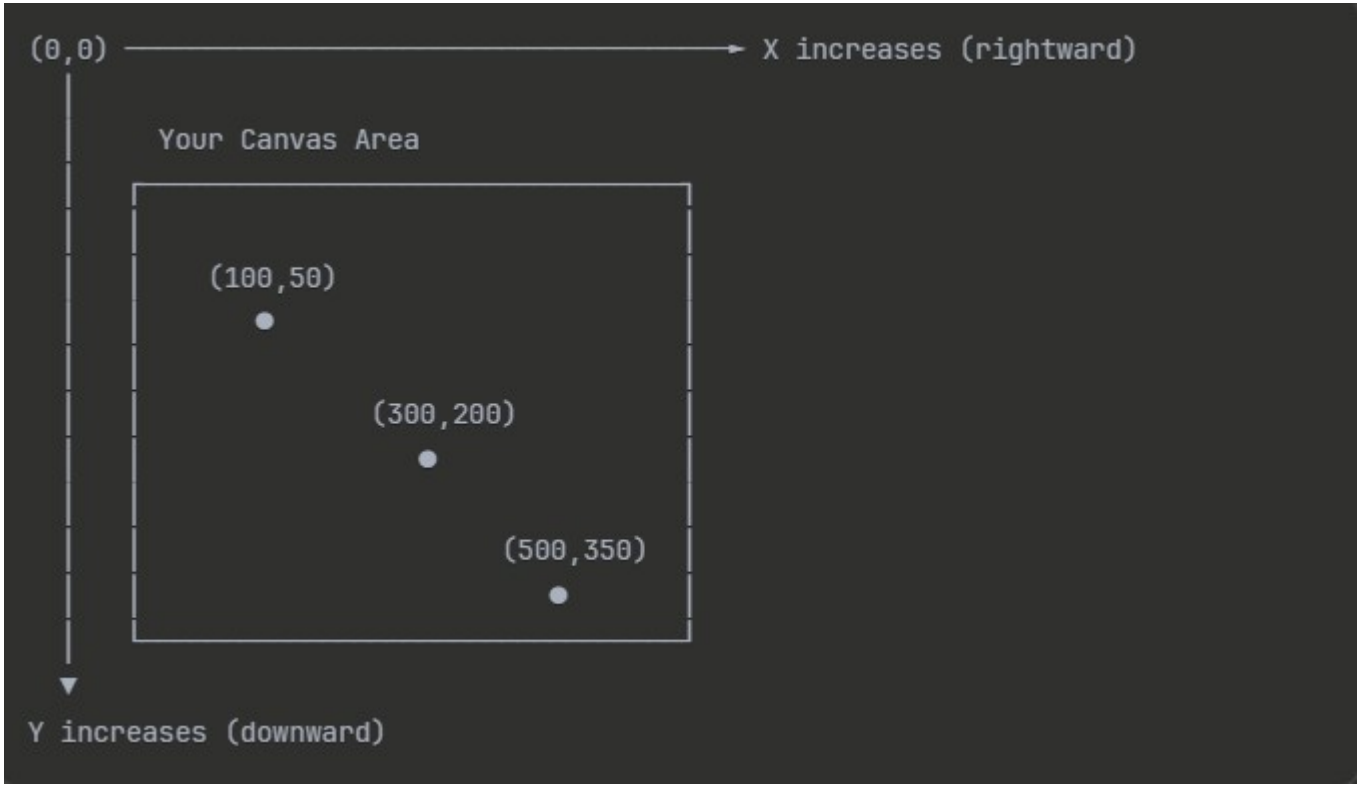
Event Loop Concept

- The mainloop() function is the heart of every GUI application. It continuously :
  - **Waits** for events (mouse clicks, key presses, window resizing)
  - **Processes** events by calling appropriate functions
  - **Updates** the display if needed
  - **Repeats** until the window is closed
- This is called event driven programming. The program responds to user actions rather than running sequentially from top to bottom

Canvas widget

- The canvas widget is a digital drawing board of sorts
- Users can create shapes, move objects and handle user interactions.
- It provides pixel-level control over what appears on screen
- Object management : Each shape gets a unique ID for easy manipulation

Understanding the Coordinate System



Coordinate system

- Key Coordinate Concepts:
- Origin Point (0,0):
  - Always at the top – left corner of the canvas
  - This is different from math class where (0,0) is often bottom left !!!
  - All other positions are measured from this point
- X – Axis (Horizontal) :
  - Increases as you move **RIGHT**
  - Negative X values are off the left edge of canvas
  - Maximum X is the canvas width

Coordinate system

- Y – Axis (Vertical) :
  - Increase as you move **DOWN**
  - Negative Y values are off the top edge of the canvas
  - Maximum Y is the canvas height

```
# create_rectangle(x1, y1, x2, y2)
# x1, y1 = top-left corner
# x2, y2 = bottom-right corner
```

Handling Keyboard Events

- Event driven programming concept :
  - Wait for something to happen (event)
  - Detect what happened (event type and details)
  - Respond by calling the appropriate function (event handler)

Creating the main window and canvas

```
# Main window
root = tk.Tk()
root.title("Keyboard Event Example")

# Canvas
canvas = tk.Canvas(root, width=400, height=300, bg="black")
canvas.pack()

# Spaceship (a rectangle here)
spaceship = canvas.create_rectangle(180, 140, 220, 160, fill="cyan")
```

Defining the movement

```
import tkinter as tk

def move_left(event):
    canvas.move(spaceship, -10, 0)

def move_right(event):
    canvas.move(spaceship, 10, 0)

def move_up(event):
    canvas.move(spaceship, 0, -10)

def move_down(event):
    canvas.move(spaceship, 0, 10)
```

Binding the functions to keys

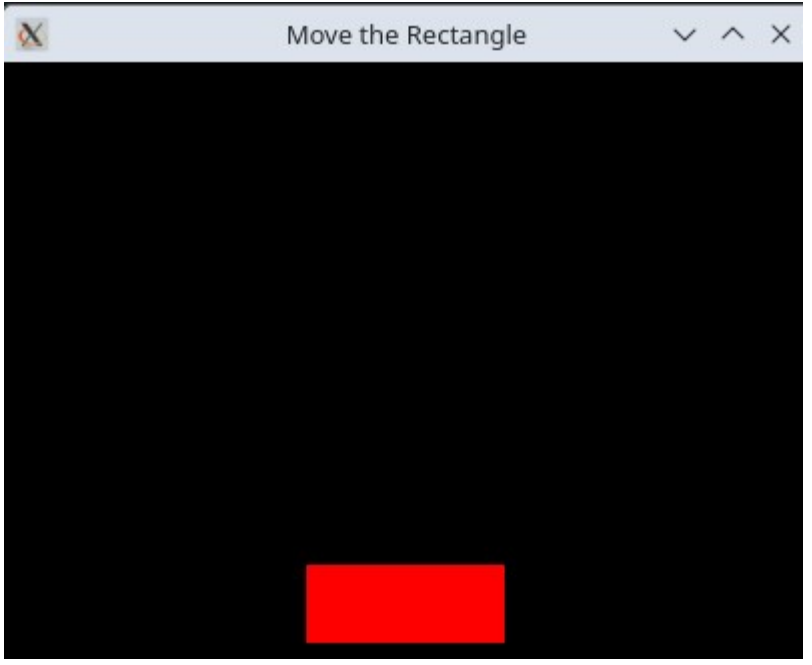
```
# Bind keys to functions
root.bind("<Left>", move_left)
root.bind("<Right>", move_right)
root.bind("<Up>", move_up)
root.bind("<Down>", move_down)

# Run
root.mainloop()
```

Binding the functions to keys

- Each function takes an event as a parameter (Tkinter automatically passes it when a key is pressed)
- Uses canvas.move(item, by x pixels, by y pixels)
- Binding Keys :
  - root.bind(“<Key>”, handler) → tells Tkinter :
    - “When this key is pressed, call that handler function
  - <Left>,<Right>,<Up>,<Down> are key event names
  - When the left arrow key is pressed the move_left() function is called

Combining it

```
import tkinter as tk

def move_rectangle(event):
    if event.keysym == 'Left':
        canvas.move(rectangle, -10, 0)  # Move left by 10 pixels
    elif event.keysym == 'Right':
        canvas.move(rectangle, 10, 0)   # Move right by 10 pixels

root = tk.Tk()
root.title("Move the Rectangle")

canvas = tk.Canvas(root, width=400, height=300, bg='black')
canvas.pack()

rectangle = canvas.create_rectangle(150, 250, 250, 290, fill='red')

root.bind('<Left>', move_rectangle)
root.bind('<Right>', move_rectangle)

canvas.focus_set()

root.mainloop()
```



What about OO?

```python
import tkinter as tk

class Spaceship:
    def __init__(self, canvas, x1, y1, x2, y2, color='blue'):
        """Initialize the spaceship (rectangle) on the canvas."""
        self.canvas = canvas
        self.spaceship = canvas.create_rectangle(x1, y1, x2, y2, fill=color)
        self.speed = 10  # Movement speed in pixels

    def move_left(self):
        """Move the spaceship left by 'speed' pixels."""
        self.canvas.move(self.spaceship, -self.speed, 0)
        # Optional: Add boundary checks to prevent moving off-screen
        if self.canvas.coords(self.spaceship)[0] < 0:
            self.canvas.move(self.spaceship, self.speed, 0)  # Bounce back

    def move_right(self):
        """Move the spaceship right by 'speed' pixels."""
        self.canvas.move(self.spaceship, self.speed, 0)
        # Optional: Add boundary checks to prevent moving off-screen
        if self.canvas.coords(self.spaceship)[2] > self.canvas.winfo_width():
            self.canvas.move(self.spaceship, -self.speed, 0)  # Bounce back
```

```python
class GameWindow:
    def __init__(self, root):
        """Initialize the game window and spaceship."""
        self.root = root
        self.root.title("Spaceship Game")

        # Create a canvas
        self.canvas = tk.Canvas(root, width=400, height=300, bg='black')
        self.canvas.pack()

        # Create a spaceship
        self.spaceship = Spaceship(self.canvas, 150, 250, 250, 290, 'red')

        # Bind arrow keys to spaceship movement
        self.root.bind('<Left>', lambda event: self.spaceship.move_left())
        self.root.bind('<Right>', lambda event: self.spaceship.move_right())

        # Focus the canvas to receive key events
        self.canvas.focus_set()

# Create the main window and start the game
root = tk.Tk()
game = GameWindow(root)
root.mainloop()
```

Summary

Any questions?